

5.1 Function Basics

So what defines a function? It has a *name* that you call it by, and a list of zero or more *arguments* or *parameters* that you hand to it for it to act on or to direct its work; it has a *body* containing the actual instructions (statements) for carrying out the task the function is supposed to perform; and it may give you back a *return value*, of a particular type.

Here is a very simple function, which accepts one argument, multiplies it by 2, and hands that value back:

```
int multbytwo(int x)
{
    int retval;
    retval = x * 2;
    return retval;
}
```

On the first line we see the return type of the function (`int`), the name of the function (`multbytwo`), and a list of the function's arguments, enclosed in parentheses. Each argument has both a name and a type; `multbytwo` accepts one argument, of type `int`, named `x`. The name `x` is arbitrary, and is used only within the definition of `multbytwo`. The caller of this function only needs to know that a single argument of type `int` is expected; the caller does not need to know what name the function will use internally to refer to that argument. (In particular, the caller does not have to pass the value of a variable named `x`.)

Next we see, surrounded by the familiar braces, the body of the function itself. This function consists of one declaration (of a local variable `retval`) and two statements. The first statement is a conventional expression statement, which computes and assigns a value to `retval`, and the second statement is a `return` statement, which causes the function to return to its caller, and also specifies the value which the function returns to its caller.

The `return` statement can return the value of any expression, so we don't really need the local `retval` variable; the function could be collapsed to

```
int multbytwo(int x)
{
    return x * 2;
}
```

How do we call a function? We've been doing so informally since day one, but now we have a chance to call one that we've written, in full detail. Here is a tiny skeletal program to call `multby2`:

```
#include <stdio.h>

extern int multbytwo(int);

int main()
{
    int i, j;
    i = 3;
    j = multbytwo(i);
    printf("%d\n", j);
    return 0;
}
```

This looks much like our other test programs, with the exception of the new line

```
extern int multbytwo(int);
```

This is an *external function prototype declaration*. It is an external declaration, in that it declares something which is defined somewhere else. (We've already seen the defining instance of the function `multbytwo`, but maybe the compiler hasn't seen it yet.) The function prototype declaration contains the three pieces of information about the function that a caller needs to know: the function's name, return type, and argument type(s). Since we don't care what name the `multbytwo` function will use to refer to its first argument, we don't need to mention it. (On the other hand, if a function takes several arguments, giving them names in the prototype may make it easier to remember which is which, so names may optionally be used in function prototype declarations.) Finally, to remind us that this is an external declaration and not a defining instance, the prototype is preceded by the keyword `extern`.

The presence of the function prototype declaration lets the compiler know that we intend to call this function, `multbytwo`. The information in the prototype lets the compiler generate the correct code for calling the function, and also enables the compiler to check up on our code (by making sure, for example, that we pass the correct number of arguments to each function we call).

Down in the body of `main`, the action of the function call should be obvious: the line

```
j = multbytwo(i);
```

calls `multbytwo`, passing it the value of `i` as its argument. When `multbytwo` returns, the return value is assigned to the variable `j`. (Notice that the value of `main`'s local variable `i` will become the value of `multbytwo`'s parameter `x`; this is absolutely not a problem, and is a normal sort of affair.)

This example is written out in "longhand," to make each step equivalent. The variable `i` isn't really needed, since we could just as well call

```
j = multbytwo(3);
```

And the variable `j` isn't really needed, either, since we could just as well call

```
printf("%d\n", multbytwo(3));
```

Here, the call to `multbytwo` is a subexpression which serves as the second argument to `printf`. The value returned by `multbytwo` is passed immediately to `printf`. (Here, as in general, we see the flexibility and generality of expressions in C. An argument passed to a function may be an arbitrarily complex subexpression, and a function call is itself an expression which may be embedded as a subexpression within arbitrarily complicated surrounding expressions.)

We should say a little more about the mechanism by which an argument is passed down from a caller into a function. Formally, C is *call by value*, which means that a function receives *copies* of the values of its arguments. We can illustrate this with an example. Suppose, in our implementation of `multbytwo`, we had gotten rid of the unnecessary `retval` variable like this:

```
int multbytwo(int x)
{
    x = x * 2;
    return x;
}
```

We might wonder, if we wrote it this way, what would happen to the value of the variable `i` when we called

```
j = multbytwo(i);
```

When our implementation of `multbytwo` changes the value of `x`, does that change the value of `i` up in the caller? The answer is no. `x` receives a copy of `i`'s value, so when we change `x` we don't change `i`.

However, there is an exception to this rule. When the argument you pass to a function is not a single variable, but is rather an array, the function does *not* receive a copy of the array, and it therefore *can* modify the array in the caller. The reason is that it might be too expensive to copy the entire array, and furthermore, it can be useful for the function to write into the caller's array, as a way of handing back more data than would fit in the function's single return value. We'll see an example of an array argument (which the function deliberately writes into) in the next chapter.

Read sequentially: [prev](#) [next](#) [up](#) [top](#)

This page by [Steve Summit](#) // [Copyright](#) 1995, 1996 // [mail](#) [feedback](#)